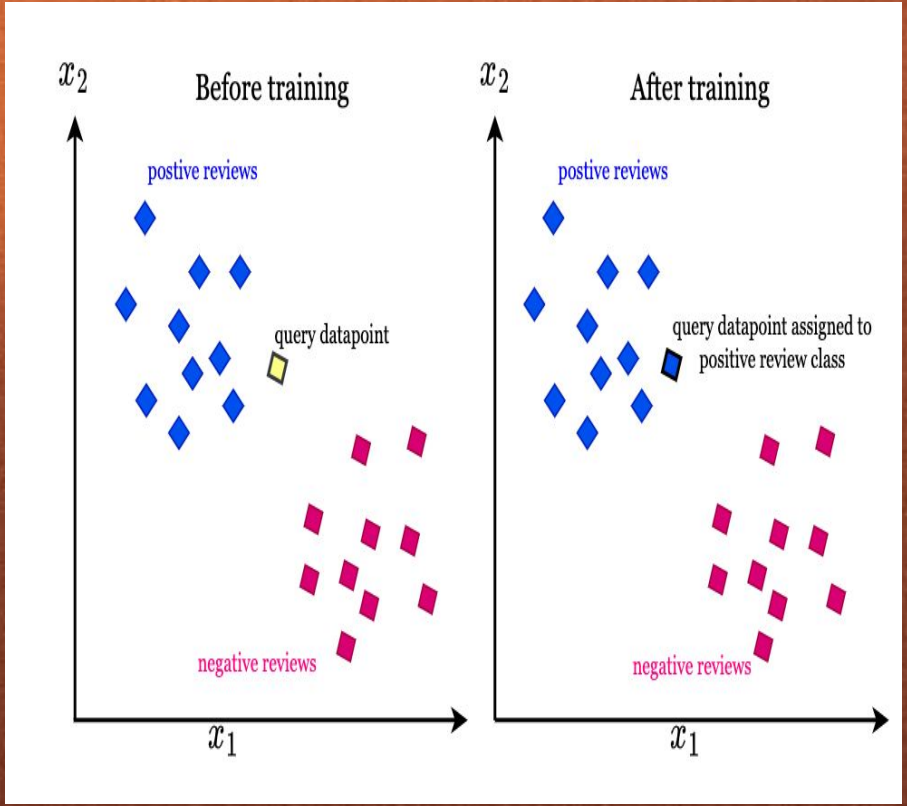
K-Nearest Neighbor Regression & Classification

Machine Learning and Data Science Club

What is the K-Nearest Neighbor Algorithm in ML?

- K-Nearest Neighbors (KNN) is a Supervised Machine Learning algorithm that makes predictions based on the most similar data points in the training set.
- It works by finding the K closest neighbors to a new data point using a distance metric, then combining their outcomes by majority vote for classification or averaging for regression.
- Because KNN relies on similarity rather than learning a model, it is simple to understand but sensitive to feature scaling and the choice of K.



An Intuitive Example

Classification vs Regression in K-NN

	Classification	Regression
Output	Class label	Continuous value
Aggregation	Majority vote	Mean / weighted mean
Examples	-Handwriting recognition -Spam detection -Disease diagnosis	-House price prediction -Recommendation scores -Demand estimation

When to employ the K-NN Algorithm

- When your dataset is not too large.
- When the relationship between features and the target is nonlinear.
- When you want a simple model that’s easy to understand.
- When all features are on a similar scale.
- When similar data points are expected to have similar outcomes.

Limitations of the K-NN Algorithm

- Can be slow when the dataset is large because it compares every point.
- Uses a lot of memory since it stores all training data.
- Performance depends heavily on the choice of K and distance metric.
- Sensitive to feature scaling — unscaled data can give poor results.
- Not ideal for real-time predictions where fast responses are needed.

Building a K-NN Model

- **Choose the value of K:**

A small K can overfit the data, while a large K can oversimplify the model.

- **Select a distance metric:**

Euclidean or Manhattan Distances

- **Find nearest neighbors**

- **Evaluate model performance:**

Use accuracy for classification and error metrics (MSE, RMSE) for regression.

Euclidean Distance

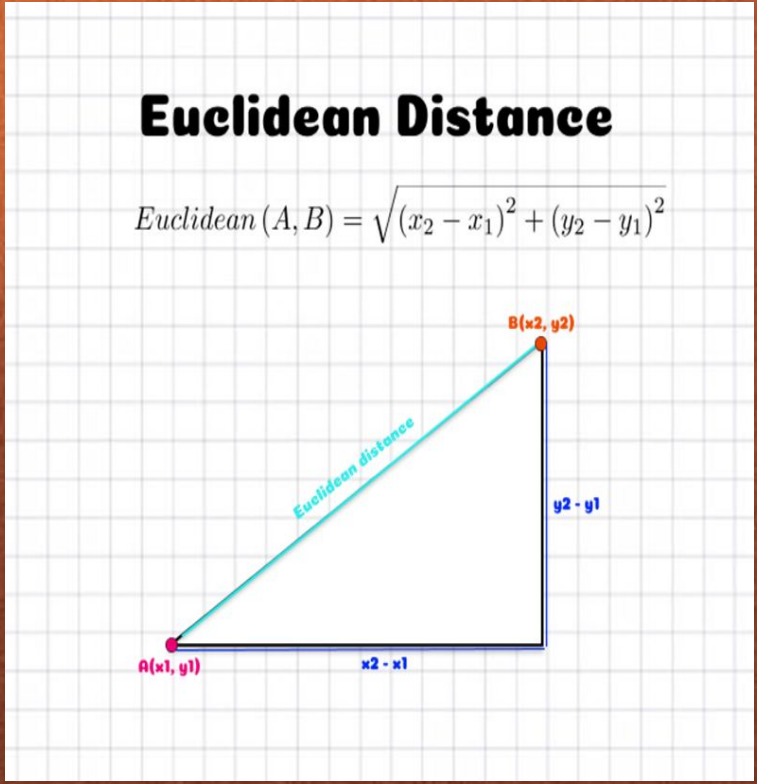
Euclidean Distance

- "Straight-line" distance between two points
- Shortest path in continuous space
- Based on geometry (like a ruler distance)

Formula:

$$d = \sqrt{\sum (x_i - y_i)^2}$$

- Sensitive to large differences (squares amplify outliers)



Manhattan Distance

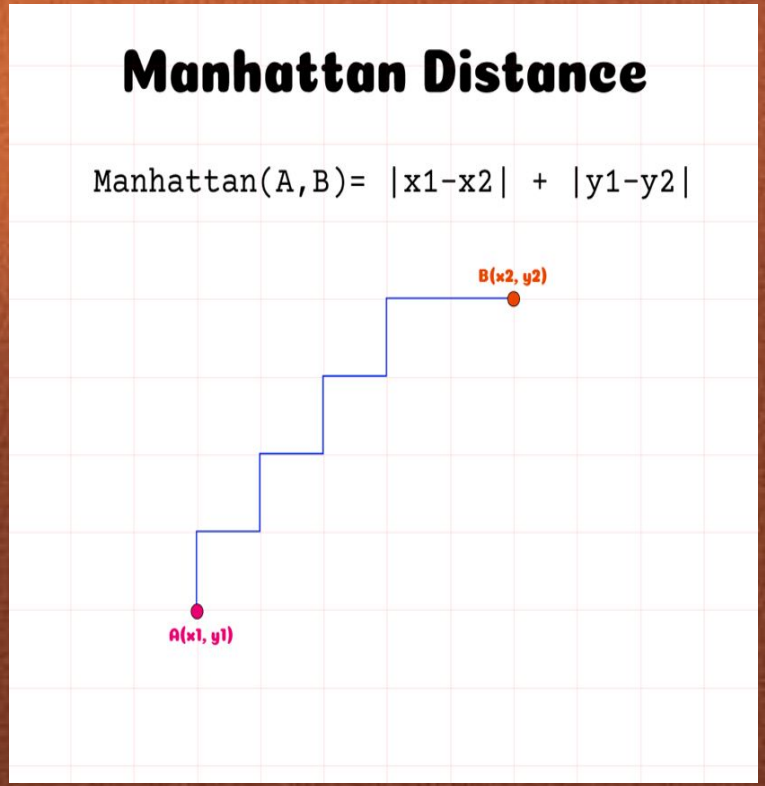
Manhattan Distance

- Grid-based" distance (like city blocks)
- Distance by moving only horizontally & vertically
- No diagonal movement

Formula:

$$d = \sum |x_i - y_i|$$

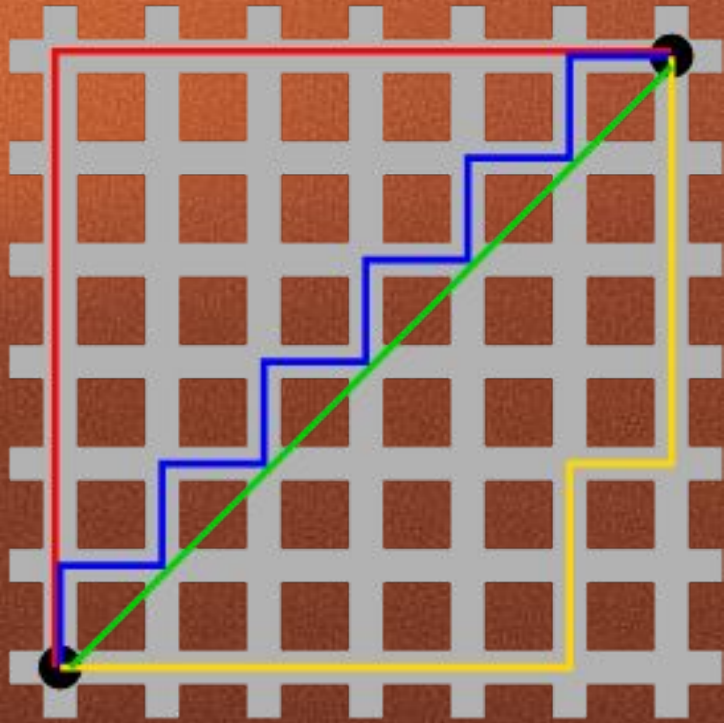
- Treats all differences linearly (no squaring)



Euclidean vs Manhattan Distance

Key Differences

- Euclidean: straight-line distance, sensitive to large differences
- Manhattan: grid-based distance, more robust to outliers

When to Use

- Euclidean: Continuous, smooth data and Features are scaled
- Manhattan: High-dimensional data and Outliers or independent features